Muhammad Shaheer | FA24-BSE-104(B)

Object Oriented Programming

Sir Shehzad Ali

COMSATS UNIVERSITY ISLAMABAD, SAHIWAL

Bank Management System using Encapsulation, Composition, and Inheritance in Java

25 October 2025

ASSIGNMENT#01

YOU ARE A SOFTWARE ENGINEER AT A BANK, AND YOUR TEAM IS RESPONSIBLE FOR DEVELOPING A **BANK MANAGEMENT SYSTEM** THAT SECURELY HANDLES CUSTOMER ACCOUNT DETAILS AND TRANSACTIONS. TO ENSURE PROPER ORGANIZATION, SECURE DATA HANDLING, AND CODE REUSABILITY, YOUR MANAGER ASKS YOU TO DESIGN THE SYSTEM USING **OBJECT-ORIENTED PROGRAMMING (OOP)** PRINCIPLES.

PART 1 – ENCAPSULATIONS

PART 2 – COMPOSITION (HAS-A RELATIONSHIP)

PART 3 – INHERITANCE

PART 4 – COMPARISON AND DISPLAY

```
import java.util.Scanner;

class BankAccount{
    private String holderName, accNum;
    private double balance;

    public BankAccount(String holderName, String accNum,
double balance) {
        this.holderName = holderName;
        this.accNum = accNum;
        this.balance = balance;
    }

    public String getHolderName() {
        return holderName;
    }

    public void setHolderName(String holderName) {
        this.holderName = holderName;
    }

    public String getAccNum() {
        return accNum;
    }

    public void setAccNum(String accNum) {
        this.accNum = accNum;
    }

    public double getBalance() {
        return balance;
```

```
    }

    public void setBalance(double balance) {
        this.balance = balance;
    }

    public void deposit(double val){
        balance +=val;
        System.out.println("Your new balance is: "+balance);
    }

    public void withdraw(double val){
        if (val%500 == 0 && val>0){
            balance-=val;
            System.out.println(val+ " withdrawn from " +
accNum + " Your remaining balance is: " + balance);
        }
        else {
            System.out.println("You can only Withdraw multiple
of 500.");
        }
    }

    public void displayAccountInfo() {
        System.out.println("Account Holder: " + holderName);
        System.out.println("Account Number: " + accNum);
        System.out.println("Balance: " + balance);
    }
}

class Customer{
    private int customerID;
    private String contactNumber;
    private BankAccount account;

    public Customer(int customerID, String contactNumber,
BankAccount account) {
        this.customerID = customerID;
        this.contactNumber = contactNumber;
        this.account = account;
    }

    public int getCustomerID() {
```

```
        return customerID;
    }

    public String getContactNumber() {
        return contactNumber;
    }

    public BankAccount getAccount() {
        return account;
    }

    public  void displayCostumer(){
        System.out.println("\n--- Customer Details ---");
        System.out.println("Customer ID: " + customerID);
        System.out.println("Contact Number: " +
contactNumber);
        account.displayAccountInfo();
    }
}

class SavingsAccount extends BankAccount{
    private final double interestRate = 0.05;

    public SavingsAccount(String holderName, String accNum,
double balance) {
        super(holderName, accNum, balance);
    }
    public void addInterest() {
        double interest = getBalance() * interestRate;
        deposit(interest);
        System.out.println("Interest added: " + interest);
    }
}

class CurrentAccount extends BankAccount{
    private final double minimumBalance = 1000;

    CurrentAccount(String holderName, String accNum, double
balance){
        super(holderName, accNum, balance);
    }
    @Override
    public void withdraw(double amount) {
```

```
        if (getBalance() - amount >= minimumBalance) {
            super.withdraw(amount);
        } else {
            System.out.println("Cannot withdraw! Must maintain
minimum balance of " + minimumBalance);
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter name for Customer 1: ");
        String name1 = sc.nextLine();
        System.out.print("Enter account number: ");
        String acc1 = sc.nextLine();
        System.out.print("Enter balance: ");
        double bal1 = sc.nextDouble();
        sc.nextLine();

        BankAccount accObj1 = new SavingsAccount(name1, acc1,
bal1);

        System.out.print("Enter Customer ID: ");
        int id1 = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter contact number: ");
        String contact1 = sc.nextLine();
        Customer c1 = new Customer(id1, contact1, accObj1);

        System.out.print("\nEnter name for Customer 2: ");
        String name2 = sc.nextLine();
        System.out.print("Enter account number: ");
        String acc2 = sc.nextLine();
        System.out.print("Enter balance: ");
        double bal2 = sc.nextDouble();
        sc.nextLine();

        BankAccount accObj2 = new CurrentAccount(name2, acc2,
bal2);
```

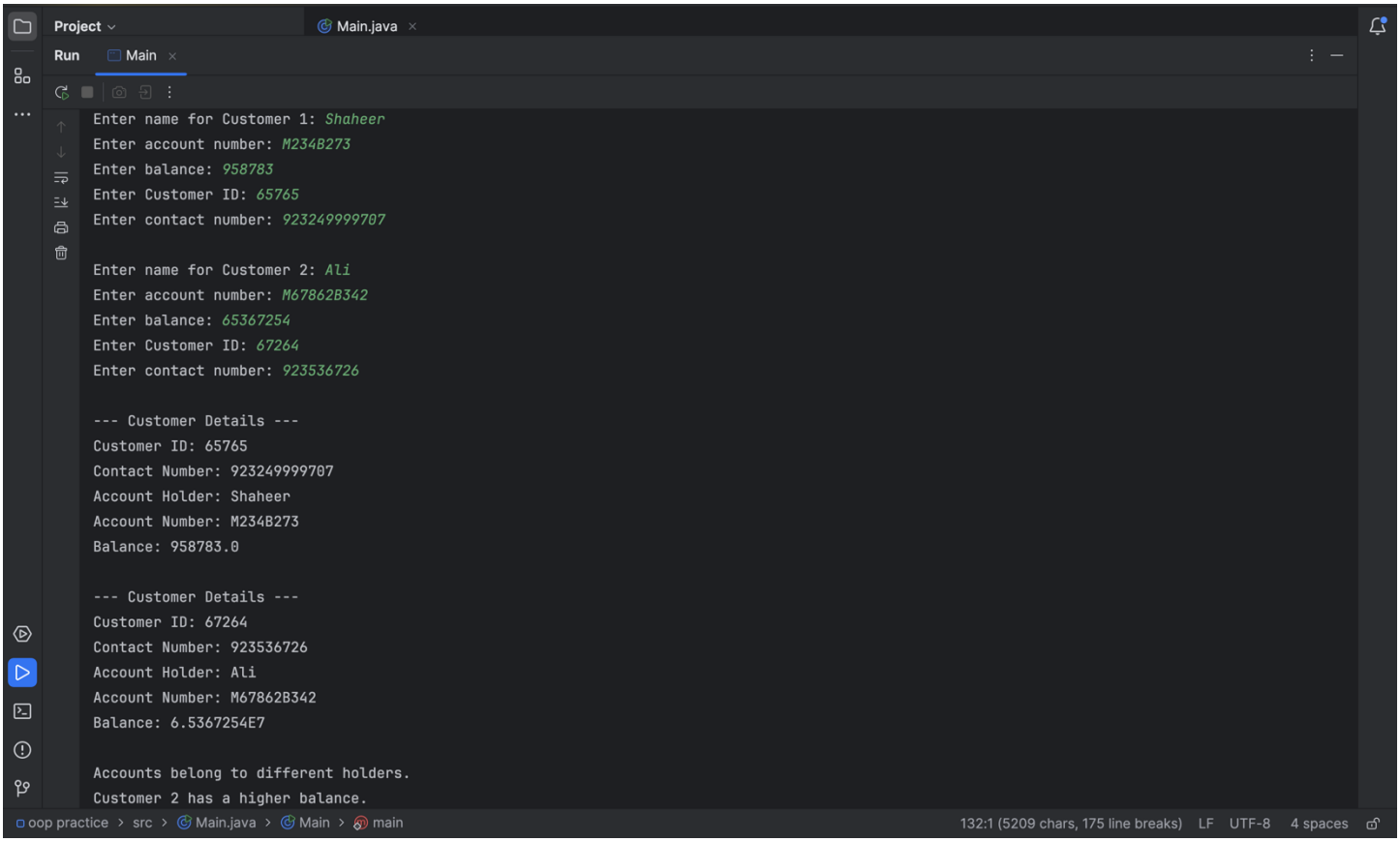
```
        System.out.print("Enter Customer ID: ");
        int id2 = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter contact number: ");
        String contact2 = sc.nextLine();
        Customer c2 = new Customer(id2, contact2, accObj2);

        c1.displayCostumer();
        c2.displayCostumer();

        if
(c1.getAccount().getHolderName().equalsIgnoreCase(c2.getAccoun
t().getHolderName())) {
            System.out.println("\nBoth accounts belong to the
same account holder.");
        } else {
            System.out.println("\nAccounts belong to different
holders.");
        }

        if (c1.getAccount().getBalance() >
c2.getAccount().getBalance()) {
            System.out.println("Customer 1 has a higher
balance.");
        } else if (c1.getAccount().getBalance() <
c2.getAccount().getBalance()) {
            System.out.println("Customer 2 has a higher
balance.");
        } else {
            System.out.println("Both customers have the same
balance.");
        }
    }
}
```


Project
Main.java
Run
Main
Enter name for Customer 1: Shaheer
Enter account number: M234B273
Enter balance: 958783
Enter Customer ID: 65765
Enter contact number: 923249999707
Enter name for Customer 2: Ali
Enter account number: M67862B342
Enter balance: 65367254
Enter Customer ID: 67264
Enter contact number: 923536726
--- Customer Details ---
Customer ID: 65765
Contact Number: 923249999707
Account Holder: Shaheer
Account Number: M234B273
Balance: 958783.0
--- Customer Details ---
Customer ID: 67264
Contact Number: 923536726
Account Holder: Ali
Account Number: M67862B342
Balance: 6.5367254E7
Accounts belong to different holders.
Customer 2 has a higher balance.
oop practice
src
Main.java
Main
main
132:1 (5209 chars, 175 line breaks)
LF
UTF-8
4 spaces